

DS 4440

(More on)

Linear Models

Last Time: The Perceptron!

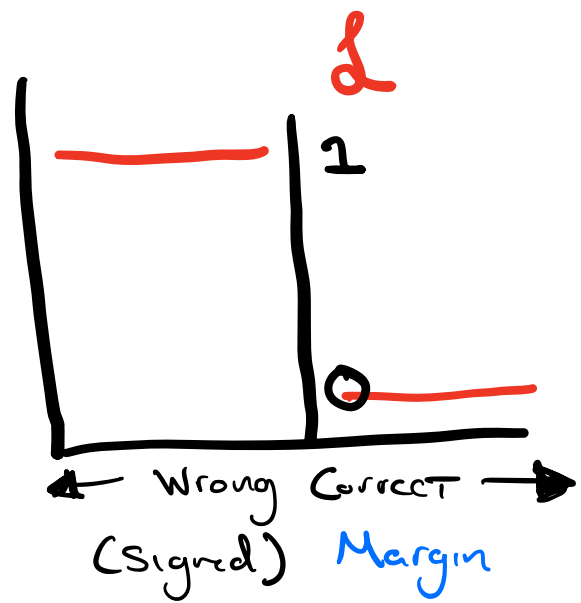
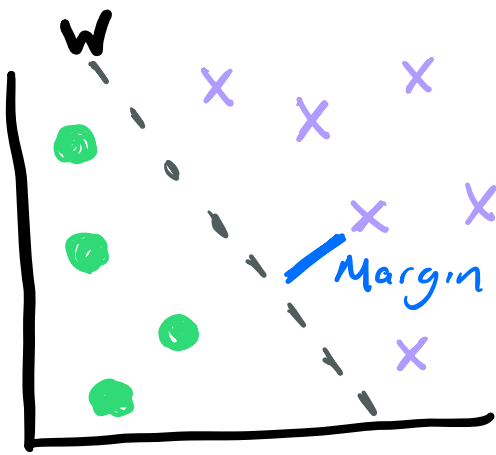
$$\hat{y}_i = \begin{cases} 1 & \text{if } W \cdot x_i > 0 \\ -1 & \text{otherwise} \end{cases}$$

★ Assumes bias terms folded into x_i, W .

Given $\langle x, y \rangle$, we fit this, i.e.,
find $\hat{W}$ to $\downarrow$ minimize a loss

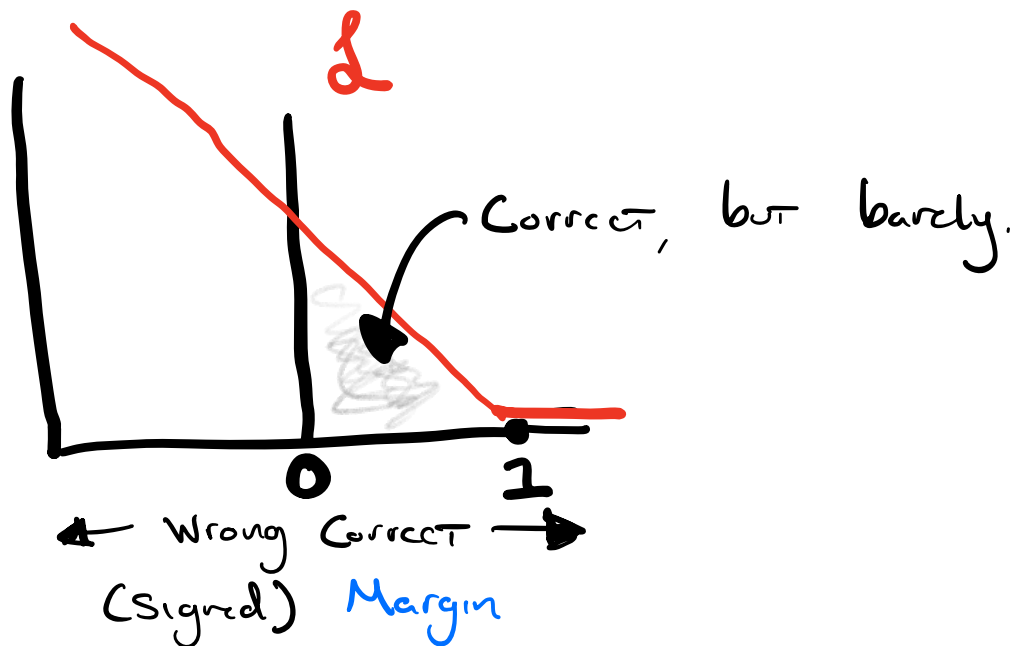
$$L(\hat{y}_i, y_i) = \begin{cases} 0 & \text{if } \hat{y}_i = y_i \\ 1 & \text{otherwise} \end{cases}$$

This is a 0/1 loss. It's simple, but not a great choice. (Why?)



One alternative: **Hinge loss**

$$L_H(\hat{w}x_i, y_i) \stackrel{\text{def}}{=} \max\{0, 1 - y_i(\hat{w}x_i)\}$$



But there are many **loss functions** we can use.

In general

$$\hat{W} \leftarrow \arg \min_W \mathcal{L}(x, y | w)$$
$$= \arg \min_W \sum_{i=1}^N \mathcal{L}(x_i, y_i | w)$$

Only minimizing loss may result in overfitting. So we often add Regularization on parameters, e.g.,

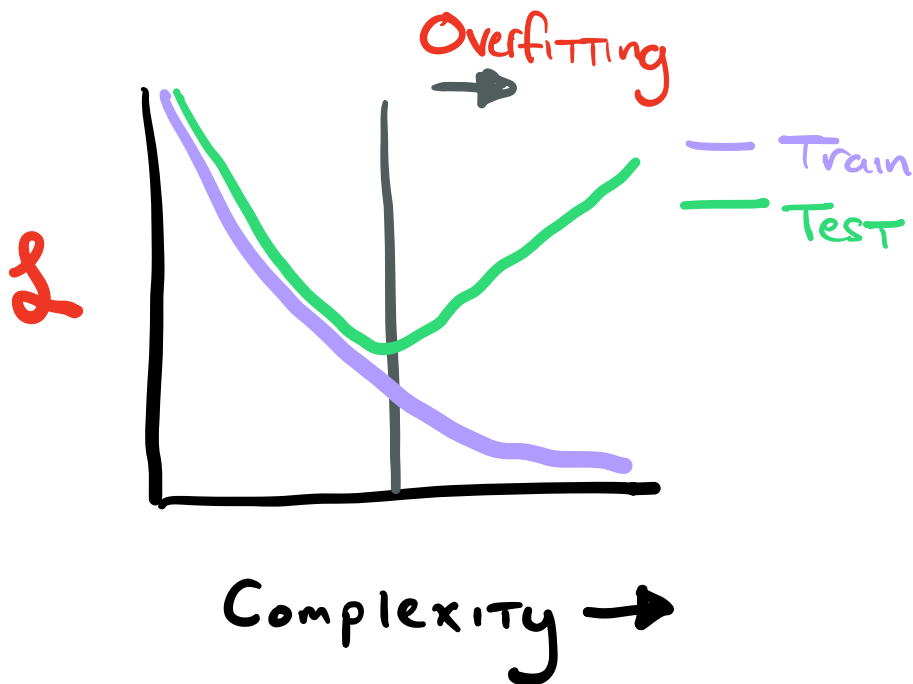
$$R(w) \stackrel{\text{def}}{=} \|w\|_2^2 = \sum_{j=1}^D w_j^2$$

Composite objective then:

$$\arg \min_W \underbrace{\mathcal{L}(x, y | w)}_{\text{Empirical loss}} + \underbrace{\lambda R(w)}_{\text{Regularizer}}$$

λ is a hyperparameter that dictates regularization "Strength".

$\uparrow \lambda$ (bias) $\rightarrow$ Lower Variance



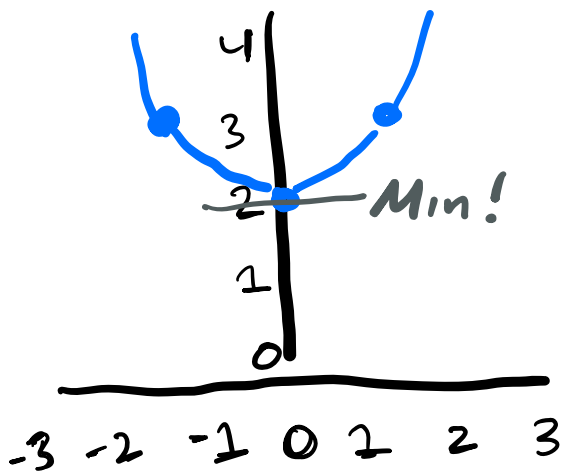
Central Q: How to find $\hat{W}$?

We did something kind of ad-hoc for Perceptron. A more general strategy — and the workhorse of Deep Learning — is gradient based

Optimization.

A quick refresher:

$$f(x) = x^2 + 2$$



Minimum Value is where

$$\frac{d}{dx} f(x) = 0$$

$$\frac{d}{dx} f(x) = 2x$$

Minimum @ $x = 0$.

Refresher! Derivative Rules

$$\frac{d}{dx} c = 0$$

$$\frac{d}{dx} x \cdot c = c$$

$$\frac{d}{dx} \ln x = \frac{1}{x}$$

$$\frac{d}{dx} x^k = k \cdot x^{(k-1)} \quad \text{Power rule}$$

$$\frac{d}{dx} [f(x) + g(x)] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x)$$

Sum rule

$$\frac{d}{dx} f(x) \cdot g(x) = \left(\frac{d}{dx} f(x) \right) \cdot g(x) + f(x) \cdot \left(\frac{d}{dx} g(x) \right)$$

Product rule

$$\frac{d}{dx} f(\underbrace{g(x)}_u) = \frac{d}{du} f(u) \cdot \frac{d}{dx} u$$

Chain rule

Gradients ▽

$$\vec{w} = [w_1 \dots w_d] \quad f(\vec{w})$$

$$\nabla_{\vec{w}} f(\vec{w}) = \left[\frac{\partial f}{\partial w_1} \dots \frac{\partial f}{\partial w_d} \right]$$

e.g. $f(\vec{w}) = w_1^3 \cdot w_2 + 5w_2 + w_1 \cdot w_3$

$$\nabla_{\vec{w}} f(\vec{w}) = [3w_1^2 w_2 + w_3, w_1^3 + 5, w_1]$$

If f maps to $\underline{m}$ outputs
we have a **Jacobian**

$$\begin{matrix} m \\ \left[\begin{array}{ccc} \frac{\partial}{\partial x_1} f_1(x) & \dots & \frac{\partial}{\partial x_d} f_1(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial}{\partial x_1} f_m(x) & \dots & \frac{\partial}{\partial x_d} f_m(x) \end{array} \right] \end{matrix} \quad \underbrace{\nabla_x f(x)}_{d\text{-dims}}$$

Useful Identities

$$A \in \mathbb{R}^{n \times m}, \quad x \in \mathbb{R}^{m \times 1} \quad \boxed{\nabla_x A x = A}$$

$$\begin{aligned} (\nabla_x A x)_i &= \nabla_x (A_{i1} x_1 + \dots + A_{im} x_m) \\ &= [A_{i1} \dots A_{im}] \end{aligned}$$

$$A \in \mathbb{R}^{n \times m}, \quad x \in \mathbb{R}^{1 \times n} \quad \boxed{\nabla_x x A = A^T}$$

$$\begin{aligned} (\nabla_x x A)_j &= \nabla_x (x_1 A_{1j} + \dots + x_n A_{nj}) \\ &= [A_{1j} \ A_{2j} \ \dots \ A_{nj}] \quad \text{jTh Col} \end{aligned}$$

$$\rightarrow \nabla_x x A = A^T$$

Elementwise Operations λ

$$y = \lambda \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix}_x = \begin{bmatrix} \lambda(x_1) \\ \lambda(x_2) \\ \vdots \\ \lambda(x_m) \end{bmatrix} \quad \nabla_x y = ?$$

$$= \begin{bmatrix} \frac{\partial}{\partial x_1} \lambda(x_1) & \dots & \frac{\partial}{\partial x_m} \lambda(x_1) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} \lambda(x_m) & \dots & \frac{\partial}{\partial x_m} \lambda(x_m) \end{bmatrix} = \begin{bmatrix} \frac{\partial}{\partial x_1} \lambda(x_1) & \dots & \frac{\partial}{\partial x_m} \lambda(x_1) \\ \vdots & & \vdots \\ \frac{\partial}{\partial x_1} \lambda(x_m) & \dots & \frac{\partial}{\partial x_m} \lambda(x_m) \end{bmatrix} \quad \emptyset$$

$$\left[\frac{\partial}{\partial x_1} \lambda(x_m) \dots \frac{\partial}{\partial x_m} \lambda(x_m) \right] \left[\phi \dots \frac{\partial}{\partial x_m} \lambda(x_m) \right]$$

Revisiting the Start of Today,
let's consider the ∇ of **Hinge**
loss + **ℓ_2 Regularizer**. (Exercise!)

$$\mathcal{L}_H(\hat{w}x_i, y_i) \stackrel{\text{def}}{=} \text{Max} \{ 0, \underline{1 - y_i(\hat{w}x_i)} \}$$

$$\nabla_w \left[\underline{1 - y_i(\hat{w}x_i)} + \lambda \frac{1}{2} \sum_j w_j^2 \right]$$

$$= \nabla_w - y_i(\hat{w}x_i) + \frac{\lambda}{2} \underbrace{\nabla_w \sum_j w_j^2}_{\substack{0 \quad \forall k \quad k \neq j \\ 2w_j \text{ when } j=k}}$$

$$= \underline{-y_i x_i} + \lambda w$$

Drops if ϕ is Max

Least Squares Regression (from an ML view)

$$\begin{matrix}
 & d & & 1 \\
 n & \begin{bmatrix} & \\ & \\ & \end{bmatrix} & n & \begin{bmatrix} & \\ & \\ & \end{bmatrix} & d & \begin{bmatrix} & \\ & \\ & \end{bmatrix} \\
 & X & & y & & w
 \end{matrix}$$

Training data

Parameters

$$\begin{aligned}
 \mathcal{L} &= \sum_{i=1}^n (y_i - \underbrace{x_i w}_{\hat{y}_i})^2 \\
 &= \|y - Xw\|^2
 \end{aligned}$$

$$\nabla_w \|y - Xw\|^2 = \left(\frac{\partial \mu}{\partial w} \right)^T \cdot \nabla_{\mu} \|\mu\|^2$$

Jacobien
incoming grad

$$= -X^T \cdot 2 \cdot u = -2X^T (y - xw)$$

$\begin{matrix} d \times n & & n \times 1 \end{matrix}$

$$= \underbrace{-2x^T y}_{\substack{d \times n \quad n \times 1}} + \underbrace{2x^T x}_{\substack{d \times d}} \underbrace{w}_{d \times 1}$$

$\frac{\quad}{d \times 1} \qquad \frac{\quad}{d \times 1}$

Set to $\vec{0}$.

$$x^T x w = x^T y$$

$$w^* = (x^T x)^{-1} x^T y$$

For LR we can solve analytically.

But not always the case!

A more flexible approach is

Gradient Descent, an iterative
optimization algorithm.

$$\hat{w}^{(t+1)} \leftarrow \hat{w}^{(t)} - \underbrace{\alpha}_{\text{learning rate}} \nabla_w \underbrace{\mathcal{L}(x, y | w^{(t)})}_{\text{usually batched}}$$

Let's see in Colab!